4

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Docker Veri Saklama ve Ağ: Volumes ve Bridge Networks

Docker Storage & Networking: Volumes and Bridge Networks

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Konteyner bir otel odası gibidir. Sabah check-out yaptığınızda temizlik görevlisi gelir ve odadaki her şeyi çöpe atar! Kalıcı eşyalarınızı otelin kiralık çelik kasasında (Volume) saklamalısınız. Odalar arası telefon hattı ise Docker Network'tür; oda numarasını (konteyner adını) çevirip doğrudan konuşursunuz.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Containers are like luxury hotel rooms. When you check out, the maid throws everything in the room into the incinerator! To keep your laptop and passport safe, you store them in the hotel's secure basement locker (Docker Volume). The intercom phone between rooms is the Docker Network; just dial the room name to talk.

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker'da üç temel veri saklama yöntemi vardır: 1) Named Volumes (Docker tarafından `/var/lib/docker/volumes` altında yönetilir; en yüksek performans ve taşınabilirlik), 2) Bind Mounts (Host makinedeki doğrudan bir klasörü `/Users/.../data` konteyner içine bağlar; geliştirme anında kod senkronizasyonu için idealdir), 3) tmpfs (Sadece RAM'de saklanır). Ağ tarafında ise varsayılan `bridge` ağı, konteynerler arasında yerleşik DNS çözünürlüğü sunar. Konteynerler birbirine IP ile değil isimleriyle (`postgres-db:5432`) erişir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Storage strategies: 1) Named Volumes (managed under `/var/lib/docker/volumes`, isolated from host OS quirks, best performance), 2) Bind Mounts (mounts arbitrary host paths, ideal for local dev live-reload), 3) tmpfs (stored strictly in host memory). On the networking side, user-defined bridge networks provide automatic embedded DNS resolution between containers by container name.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# 1. Özel bir bridge ağı oluştur
docker network create coffee-net

# 2. Kalıcı bir veri diski (Volume) oluştur
docker volume create postgres-data

# 3. PostgreSQL'i ağa ve diske bağlayarak başlat
docker run -d --name postgres-db --network coffee-net \
  -v postgres-data:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=coffeedb postgres:16-alpine

# 4. Spring Boot'u aynı ağa bağla (IP yerine doğrudan 'postgres-db' adını kullanır!)
docker run -d --name coffee-api --network coffee-net -p 8080:8080 \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://postgres-db:5432/coffeedb \
  ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Varsayılan (default) bridge ağında konteyner isimleriyle otomatik DNS çözümlemesi çalışmaz! İsimle haberleşme için mutlaka `docker network create` ile kendi özel ağınızı oluşturmalısınız.

🇬🇧 WATCH OUT

The default legacy docker0 bridge does NOT provide automatic DNS resolution by container name! Always create a user-defined bridge network.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Named Volumes konteyner silinse bile veritabanı verilerini korur.
- Docker yerel DNS'i sayesinde konteynerler IP değişse bile birbirini bulur.
- Veritabanı ve Spring Boot uygulamalarını aynı özel ağa bağlamak şarttır.

- Named volumes persist state even when containers are stopped and destroyed.
- Docker embedded DNS resolves container names dynamically without static IPs.
- Co-locating apps and databases on dedicated networks isolates traffic securely.